Slides for P3666R3
Bit-precise integers

Document number:
P4157R0

Date:
2026-03-26

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:
Jan Schultke <janschultke@gmail.com>

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/bitint-lewg-slides.cow

---

> **IMPORTANT:** This document has custom controls:
>
> - →, ↓: go to the next slide
> - ←, ↑: go to previous slide

Bit-precise integers

P3666R3

Introduction

C23 now has `_BitInt` type for N-bit integers (WG14 N2763, N2775):

```
// 8-bit unsigned integer initialized with value 255.
// The literal suffix wb is unnecessary in this case.
unsigned _BitInt(8) x = 0xFFwb;
```

- implemented by GCC and Clang; max: `_BitInt(8'388'608)`
- forwarded by EWG to LEWG/CWG for C++29

P3666 Library decisions to be made

- How much library support to provide?
  - Aggressively minimal: just the core feature
  - Minimal but useful: P3666R3
  - More extensive: `<simd>`, `<atomic>`, etc.
- Add `std::bit_int` and `std::bit_uint` alias templates?
- Is `std::is_integral_v<_BitInt(N)>` true?

Anecdote: `std::cmp_less`

```cpp
template<typename _Tp, typename _Up>
constexpr bool cmp_less(_Tp __t, _Up __u) noexcept
{
  static_assert(__is_signed_or_unsigned_integer<_Tp>::value);
  static_assert(__is_signed_or_unsigned_integer<_Up>::value);

  if constexpr (is_signed_v<_Tp> == is_signed_v<_Up>)
    return __t < __u;
  else if constexpr (is_signed_v<_Tp>)
    return __t < 0 || make_unsigned_t<_Tp>(__t) < __u;
  else
    return __u >= 0 && __t < make_unsigned_t<_Up>(__u);
}
```

Alias templates

```cpp
template<size_t N>
  using bit_int = _BitInt(N);

template<size_t N>
  using bit_uint = unsigned _BitInt(N);
```

- deduction is supported
- library spec should prefer alias templates over keyword spelling

`std::is_integral_v<_BitInt(N)>`

```
integral types
    ├── signed or unsigned integer types
    │    ├── standard integers: int, unsigned, etc.
    │    ├── extended integers: __int128 etc.
    │    └── bit-precise integers: (unsigned) _BitInt(N)
    ├── character types: char, wchar_t, etc.
    └── bool
```

- mostly matches C2y taxonomy
- *very surprising* if `std::is_integral_v<_BitInt(N)>` was `false`
- huge wording/teaching effort to treat `_BitInt(N)` specially